

SWE308: Information Security

Lecture 7: Symmetric Cryptography - Block Ciphers

9a c3 49 72 10 d9 17 12 b6 34 08 bc 7c 60 90 d9

Dr Iftekhar Salam
iftekhar.salam@xmu.edu.my

Xiamen Univesity Malaysia

Presentation Outline

- 1 What is Block Cipher and How it Works?
- 2 Example of Block Ciphers
- 3 Advanced Encryption Standard (AES)
- 4 Block Cipher Modes of Operation

Presentation Outline

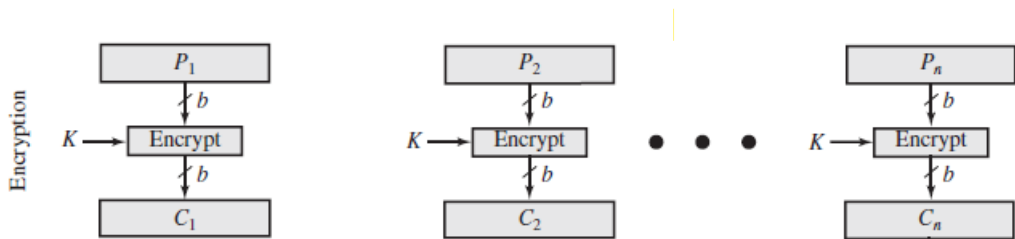
- 1 What is Block Cipher and How it Works?
- 2 Example of Block Ciphers
- 3 Advanced Encryption Standard (AES)
- 4 Block Cipher Modes of Operation

Block Ciphers

Plaintext is encrypted one block at a time

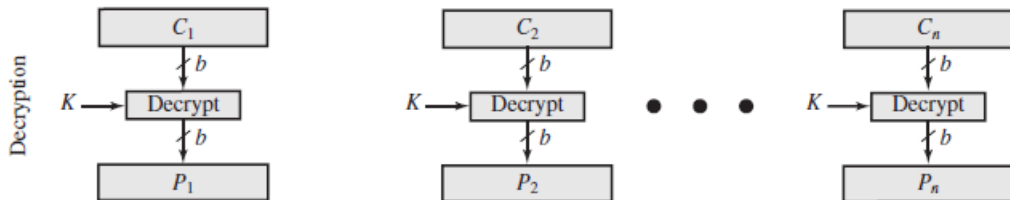
- Block size is usually larger than a single character,
 - Common block sizes are 64-bits or 128-bits
- In the basic block cipher mode of operation,
 - Both the plaintext block and the key are input to the encryption algorithm
 - The output of the encryption algorithm is one block of ciphertext
- Ciphertext is decrypted one block at a time
 - produces a block of plaintext each time

Block Ciphers: Basic Encryption Operation



- The plaintext is divided into blocks $P_1 P_2 P_3 \dots$
- Each block of plaintext is encrypted (using the key K) to produce the corresponding ciphertext blocks $C_1 C_2 C_3 \dots$

Block Ciphers: Basic Decryption Operation



- The ciphertext is divided into blocks $C_1 C_2 C_1 \dots$
- Each block of ciphertext is decrypted (using the same key K) to produce the corresponding plaintext block $P_1 P_2 P_3 \dots$
- Then the plaintext is reassembled.

Shannon's Concepts: Confusion and Diffusion

Claude Shannon, in his 1945 classified report "A Mathematical Theory of Cryptography"¹

- introduced two properties of the operation of a secure cipher
- confusion and diffusion

Confusion

- To make the relation between the ciphertext and the key a very complex and involved one
- Each bit of the ciphertext block has highly nonlinear relations with the key bits.

Diffusion

- Each plaintext block bit or key bit affects many bits of the ciphertext block

Avalanche effect

- small modification, big impact

¹<https://www.iacr.org/museum/shannon45.html>

Shannon's Concept: Confusion and Diffusion

Consider x, y, k represents the plaintext, ciphertext, key, respectively (8-bit long)

- Good diffusion: Each plaintext bits (and the key) affects half of the output bits in y_i
- Bad confusion: The relationship is simple linear functions

$$y_1 = x_1 + x_2 + x_3 + x_4 + k_1 + k_2 + k_3 + k_4$$

$$y_2 = x_2 + x_3 + x_4 + x_5 + k_2 + k_3 + k_4 + k_5$$

$$y_3 = x_3 + x_4 + x_5 + x_6 + k_3 + k_4 + k_5 + k_6$$

$$y_4 = x_4 + x_5 + x_6 + x_7 + k_4 + k_5 + k_6 + k_7$$

$$y_5 = x_5 + x_6 + x_7 + x_8 + k_5 + k_6 + k_7 + k_8$$

$$y_6 = x_6 + x_7 + x_8 + x_1 + k_6 + k_7 + k_8 + k_1$$

$$y_7 = x_7 + x_8 + x_1 + x_2 + k_7 + k_8 + k_1 + k_2$$

$$y_8 = x_8 + x_1 + x_2 + x_3 + k_8 + k_1 + k_2 + k_3$$

The encryption and decryption functions of a cipher should have both good confusion and diffusion of the message block bits and secret key bits.

Presentation Outline

- 1 What is Block Cipher and How it Works?
- 2 Example of Block Ciphers
- 3 Advanced Encryption Standard (AES)
- 4 Block Cipher Modes of Operation

Commonly used Block Cipher Example

Data Encryption Standard (DES):

- Actually, Data Encryption Algorithm (DEA)
- Published in 1977 by US National Bureau of Standards as FIPS publication 46
 - That is, this is a US standard, but a defacto global standard
- Plaintext and ciphertext are 64 bit blocks
- Key size: 56 bits
- This key size is small enough that DES is considered insecure (In 1999, break in 22 hours)
 - Triple-DES (3DES) provides a way to extend the useful life of DES, approved by the FIPS

A More Recent Block Cipher Example

Advanced Encryption Standard (AES)

- 1997: US National Institute of Standards and Technology (NIST) made a call for submissions to establish a standard block cipher more secure than DES
 - 15 designs submitted
- 2001: **Rijndael** selected as the winner [developed by J. Daemen and V. Rijmen (Belgian cryptographers)]
- Plaintext and Ciphertext are 128-bit blocks
- Variable key size: 128, 192, or 256-bit keys allowed

Presentation Outline

- 1 What is Block Cipher and How it Works?
- 2 Example of Block Ciphers
- 3 Advanced Encryption Standard (AES)
- 4 Block Cipher Modes of Operation

Advanced Encryption Standard (AES)

NIST's current standard for block cipher

AES variant	Key length	No of Rounds
AES-128	128	10
AES-192	192	12
AES-256	256	14

Substitution Permutation Network (SPN) with each round having

- Randomness layer: Round key addition
- Confusion layer: Byte substitution
- Diffusion layer: Shift row and Mix column (the last round does not have the mix column step)

The AES State Representation

Considered each 128-bit block as consisting of a 4×4 array/matrix of bytes, arranged as follows

- Notice that the first four bytes of a 128-bit input block occupy the first column in the 4×4 array of bytes. The next four bytes occupy the second column, and so on

16 byte plaintext

a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

a	e	i	m
b	f	j	n
c	g	k	o
d	h	l	p



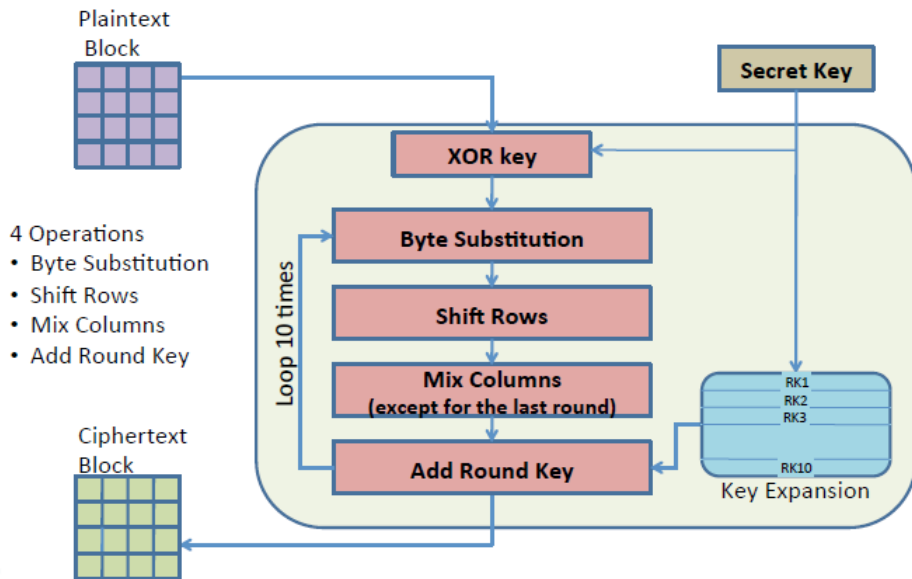
AES



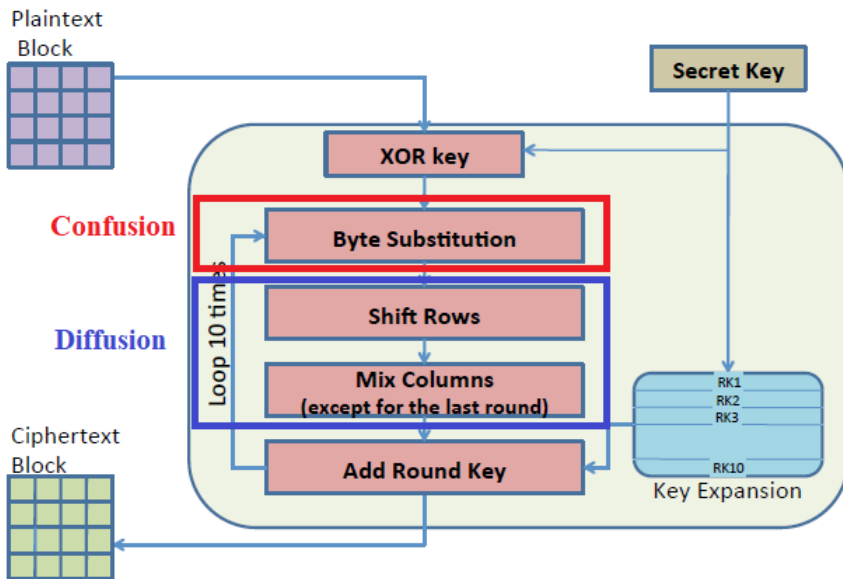
A	E	I	M
B	F	J	N
C	G	K	O
D	H	L	P

16 byte ciphertext

AES-128 Encryption - A High Level View



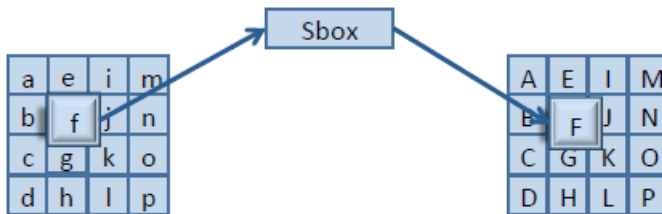
AES-128 Encryption - A High Level View



Byte Substitution

SubBytes

- This step consists of using a 16×16 lookup table (S-box) to find a replacement byte for a given byte in the input state array
- The entries in the lookup table are created by using the notions of multiplicative inverses in $GF(2^8)$ and bit scrambling to destroy the bit-level correlations inside each byte (the details outside the scope of this course) ²
- S-box is the only source of non-linearity in AES



²GF stands for Galois field

S-box Encryption Table

AES S-Box

- The column is determined by the least significant nibble, and the row by the most significant nibble

	00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	0e	0f
00	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
10	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
20	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
30	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
40	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
50	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
60	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
70	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
80	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
90	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
a0	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
b0	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
c0	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
d0	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
e0	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
f0	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

S-box Encryption Table

AES S-Box

- The column is determined by the least significant nibble, and the row by the most significant nibble

	00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	0e	0f
00	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
10	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
20	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
30	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
40	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
50	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
60	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
70	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
80	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
90	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
a0	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
b0	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
c0	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
d0	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
e0	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
f0	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

For example, the value **0x42** is converted to **0x2c**

S-box Decryption Table

AES Inverse S-Box

- The column is determined by the least significant nibble, and the row by the most significant nibble

	00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	0e	0f
00	52	09	6a	d5	30	36	a5	38	bf	40	a3	9e	81	f3	d7	fb
10	7c	e3	39	82	9b	2f	ff	87	34	8e	43	44	c4	de	e9	cb
20	54	7b	94	32	a6	c2	23	3d	ee	4c	95	0b	42	fa	c3	4e
30	08	2e	a1	66	28	d9	24	b2	76	5b	a2	49	6d	8b	d1	25
40	72	f8	f6	64	86	68	98	16	d4	a4	5c	cc	5d	65	b6	92
50	6c	70	48	50	fd	ed	b9	da	5e	15	46	57	a7	8d	9d	84
60	90	d8	ab	00	8c	bc	d3	0a	f7	e4	58	05	b8	b3	45	06
70	d0	2c	1e	8f	ca	3f	0f	02	c1	af	bd	03	01	13	8a	6b
80	3a	91	11	41	4f	67	dc	ea	97	f2	cf	ce	f0	b4	e6	73
90	96	ac	74	22	e7	ad	35	85	e2	f9	37	e8	1c	75	df	6e
a0	47	f1	1a	71	1d	29	c5	89	6f	b7	62	0e	aa	18	be	1b
b0	fc	56	3e	4b	c6	d2	79	20	9a	db	c0	fe	78	cd	5a	f4
c0	1f	dd	a8	33	88	07	c7	31	b1	12	10	59	27	80	ec	5f
d0	60	51	7f	a9	19	b5	4a	0d	2d	e5	7a	9f	93	c9	9c	ef
e0	a0	e0	3b	4d	ae	2a	f5	b0	c8	eb	bb	3c	83	53	99	61
f0	17	2b	04	7e	ba	77	d6	26	e1	69	14	63	55	21	0c	7d

S-box Decryption Table

AES Inverse S-Box

- The column is determined by the least significant nibble, and the row by the most significant nibble

	00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	0e	0f
00	52	09	6a	d5	30	36	a5	38	bf	40	a3	9e	81	f3	d7	fb
10	7c	e3	39	82	9b	2f	ff	87	34	8e	43	44	c4	de	e9	cb
20	54	7b	94	32	a6	c2	23	3d	ee	4c	95	0b	42	fa	c3	4e
30	08	2e	a1	66	28	d9	24	b2	76	5b	a2	49	6d	8b	d1	25
40	72	f8	f6	64	86	68	98	16	d4	a4	5c	cc	5d	65	b6	92
50	6c	70	48	50	fd	ed	b9	da	5e	15	46	57	a7	8d	9d	84
60	90	d8	ab	00	8c	bc	d3	0a	f7	e4	58	05	b8	b3	45	06
70	d0	2c	1e	8f	ca	3f	0f	02	c1	af	bd	03	01	13	8a	6b
80	3a	91	11	41	4f	67	dc	ea	97	f2	cf	ce	f0	b4	e6	73
90	96	ac	74	22	e7	ad	35	85	e2	f9	37	e8	1c	75	df	6e
a0	47	f1	1a	71	1d	29	c5	89	6f	b7	62	0e	aa	18	be	1b
b0	fc	56	3e	4b	c6	d2	79	20	9a	db	c0	fe	78	cd	5a	f4
c0	1f	dd	a8	33	88	07	c7	31	b1	12	10	59	27	80	ec	5f
d0	60	51	7f	a9	19	b5	4a	0d	2d	e5	7a	9f	93	c9	9c	ef
e0	a0	e0	3b	4d	ae	2a	f5	b0	c8	eb	bb	3c	83	53	99	61
f0	17	2b	04	7e	ba	77	d6	26	e1	69	14	63	55	21	0c	7d

For example, the value **0x2c** is converted back to **0x42**

ShiftRows

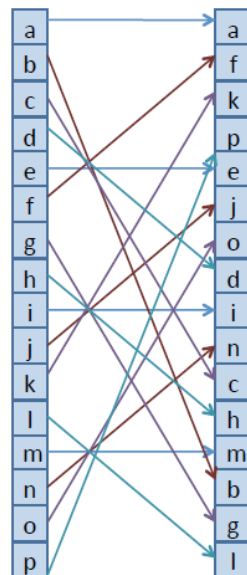
Diffusion layer

- First row: no changes
- Left rotate
 - (2nd Row by 8 bits)
- Left rotate
 - (3rd Row by 16 bits)
- Left rotate
 - (4th Row by 24 bits)

a	e	i	m
b	f	j	n
c	g	k	o
d	h	l	p



a	e	i	m
f	j	n	b
k	o	c	g
p	d	h	l

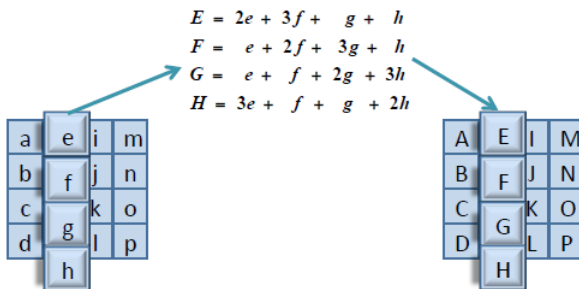


The goal of this transformation is to scramble the byte order inside each 128-bit block

MixColumns

- The 4×4 matrix is multiplied with a fixed matrix
- Further scramble up the 128-bit input block
- If the input of a column changes then all outputs change

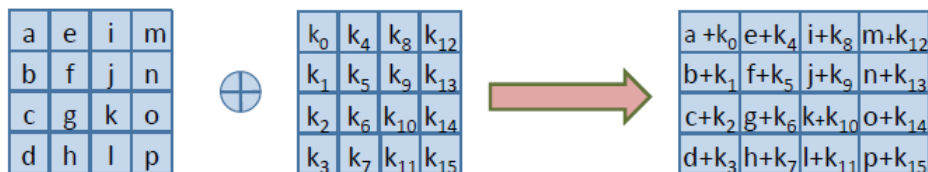
$$\begin{bmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{bmatrix} \times \begin{bmatrix} a & e & i & m \\ b & f & j & n \\ c & g & k & o \\ d & h & l & p \end{bmatrix}$$



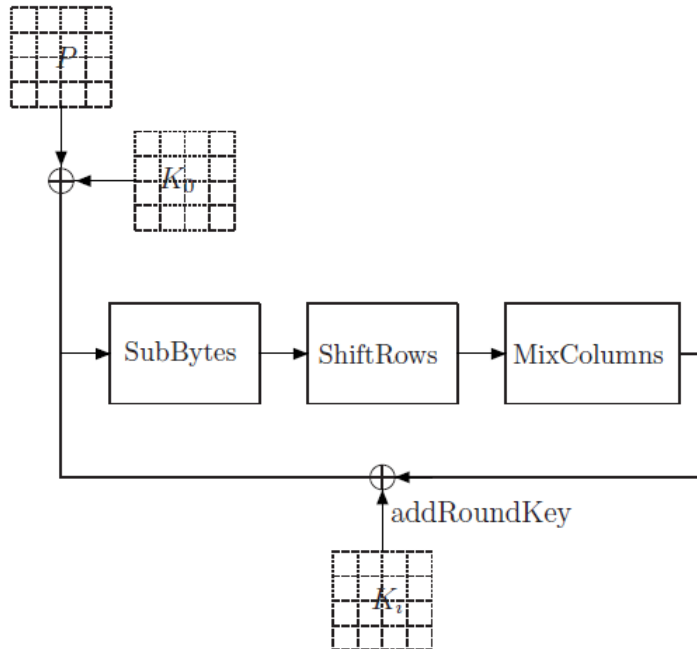
The shift-rows step along with the mix-column step causes each bit of the ciphertext to depend on every bit of the plaintext after 10 rounds of processing

AddRoundKey

- Adding the round key to the output of the previous step during the forward process
- Addition here is addition in $GF(2^8)$, which is the XOR operation



AES-128 Encryption - A Top Level View



Presentation Outline

- 1 What is Block Cipher and How it Works?
- 2 Example of Block Ciphers
- 3 Advanced Encryption Standard (AES)
- 4 Block Cipher Modes of Operation

Modes of Operation

Block ciphers can be used in different modes to provide different security services.

Common modes of operation:

- Electronic Code Book (ECB)
- Cipher Block Chaining (CBC)
- Output Feedback (OFB)
- Cipher Feedback (CFB)
- Cipher Block Chaining Message Authentication Code (CBC-MAC)
- Counter with Cipher Block Chaining Message Authentication Code (CCM)
- Offset Codebook Mode (OCB)
- Galois Counter Mode (GCM)
- ...

NIST currently has 14 approved modes of operation³

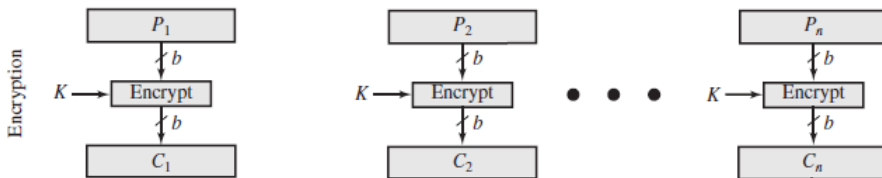
- 8 for confidentiality
- Others for authentication and key-wrapping

³[https://csrc.nist.gov/projects/block-cipher-techniques/bcm#:~:text=Block%20Cipher%20Modes&text=As%20summarized%20on%20the%20Current,%2C%20KWP%2C%20and%20TKW\).](https://csrc.nist.gov/projects/block-cipher-techniques/bcm#:~:text=Block%20Cipher%20Modes&text=As%20summarized%20on%20the%20Current,%2C%20KWP%2C%20and%20TKW).)

Electronic Code Book (ECB)

Electronic CodeBook (ECB) mode encryption

- The simplest way to proceed is what is known as electronic codebook (ECB) mode
- Plaintext is handled b bits at a time and each block is encrypted using the same key
- Plaintext data is divided into blocks $P_1, P_2, \dots, P_n$ (last block is padded as required)
- Each block is then processed separately

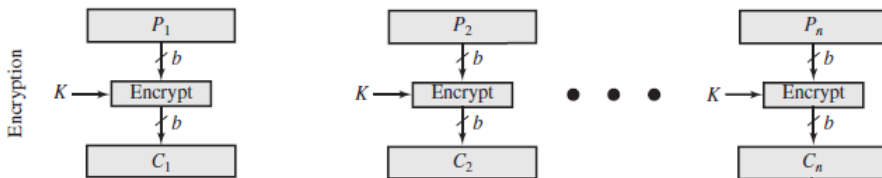


- Encryption parallelizable?

Electronic Code Book (ECB)

Electronic CodeBook (ECB) mode encryption

- The simplest way to proceed is what is known as electronic codebook (ECB) mode
- Plaintext is handled b bits at a time and each block is encrypted using the same key
- Plaintext data is divided into blocks $P_1, P_2, \dots, P_n$ (last block is padded as required)
- Each block is then processed separately

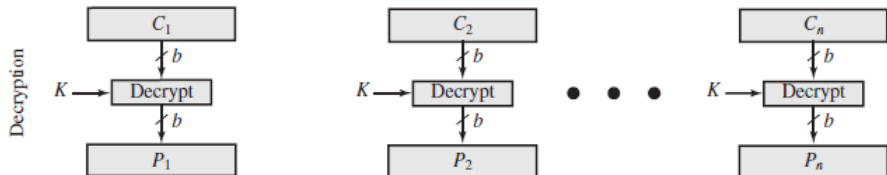


- Encryption parallelizable?
 - Yes

Electronic Code Book (ECB)

Electronic CodeBook (ECB) mode decryption

- Ciphertext is handled b bits at a time and each block is decrypted using the same key
- Ciphertext data is divided into blocks (of b bits) $C_1, C_2, \dots, C_n$
- Each block is then processed separately
 - Ciphertext block and key used as inputs to the decryption algorithm
 - Need to use same key as used for encryption (symmetric cipher)
 - Recovered plaintext is assembled by concatenating blocks, removing padding if required

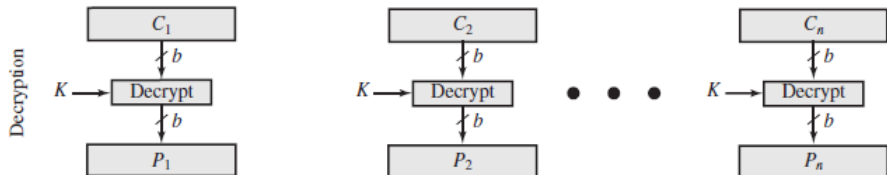


- Decryption parallelizable?

Electronic Code Book (ECB)

Electronic CodeBook (ECB) mode decryption

- Ciphertext is handled b bits at a time and each block is decrypted using the same key
- Ciphertext data is divided into blocks (of b bits) $C_1, C_2, \dots, C_n$
- Each block is then processed separately
 - Ciphertext block and key used as inputs to the decryption algorithm
 - Need to use same key as used for encryption (symmetric cipher)
 - Recovered plaintext is assembled by concatenating blocks, removing padding if required



- Decryption parallelizable?
 - Yes

Issues with ECB Mode?

What happens if the ciphertext is modified?

- Bitflip error (0 to 1 or vice versa): that block will decrypt incorrectly
- Ciphertext bit (or even a character) inserted or deleted will be detected because of the incorrect ciphertext length
 - But ECB mode cannot detect insertion or deletion in multiples of block size

For a given key, same plaintext block always encrypts to same ciphertext block

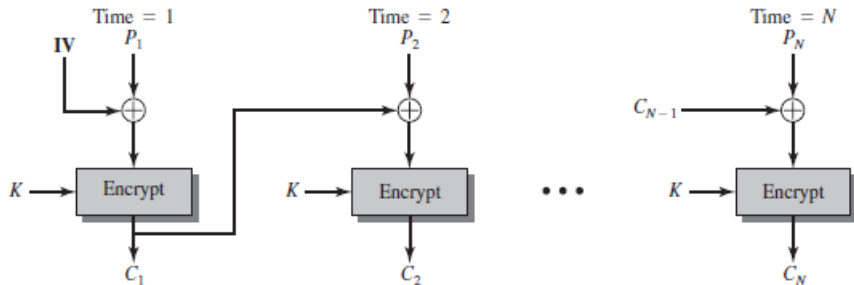
- Does not hide repetitive data well.
- Attacker may construct a code book of known plaintext/ciphertext blocks and use this codebook to insert, delete, reorder or replay blocks within the message
- If the message is highly structured, it may be possible for a cryptanalyst to exploit these regularities

Increase security by finding other modes of operation so that if same plaintext block is encrypted multiple times, the ciphertext formed may be different each time

Cipher Block Chaining(CBC)

Cipher Block Chaining (CBC) mode encryption

- Plaintext data is divided into blocks $P_1, P_2, \dots, P_n$ (last block may require padding)
- The encryption algorithm now takes three inputs:
 - the key, the plaintext block and the previous ciphertext block
 - For the first plaintext block, an **initialisation vector (IV)** is used to ensure that two encryptions of the same plaintext will result in different ciphertext
- Encryption:**
 - $C_1 = E(K, [P_1 \oplus IV])$
 - $C_i = E(K, [P_i \oplus C_{i-1}])$

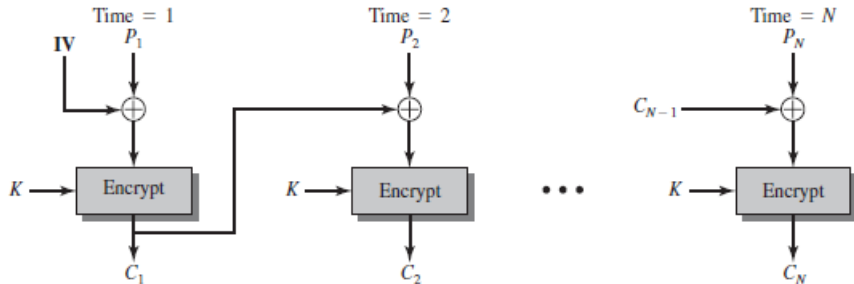


- Encryption parallelizable?

Cipher Block Chaining(CBC)

Cipher Block Chaining (CBC) mode encryption

- Plaintext data is divided into blocks $P_1, P_2, \dots, P_n$ (last block may require padding)
- The encryption algorithm now takes three inputs:
 - the key, the plaintext block and the previous ciphertext block
 - For the first plaintext block, an initialisation vector (IV) is used to ensure that two encryptions of the same plaintext will result in different ciphertext
- Encryption:**
 - $C_1 = E(K, [P_1 \oplus IV])$
 - $C_i = E(K, [P_i \oplus C_{i-1}])$

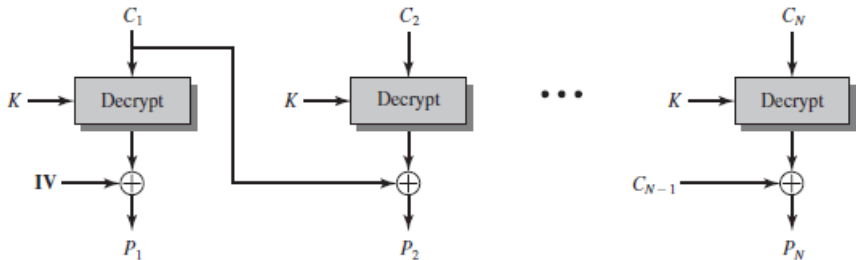


- Encryption parallelizable?**
 - No

Cipher Block Chaining(CBC)

Cipher Block Chaining (CBC) mode decryption

- Ciphertext data is received as blocks $C_1, C_2, \dots, C_n$
- The decryption algorithm now takes three inputs:
 - the key, the ciphertext block and the previous ciphertext block
 - For the first ciphertext block, the same initialisation vector (IV) from the sender side is used
- **Decryption:**
 - $P_1 = D(K, C_1) \oplus IV$
 - $P_i = D(K, C_i) \oplus C_{i-1}$

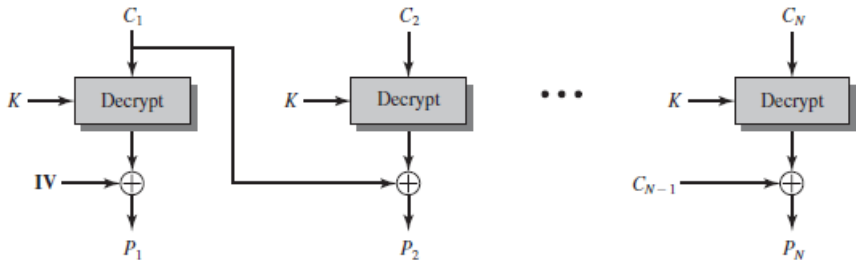


- Decryption parallelizable?

Cipher Block Chaining(CBC)

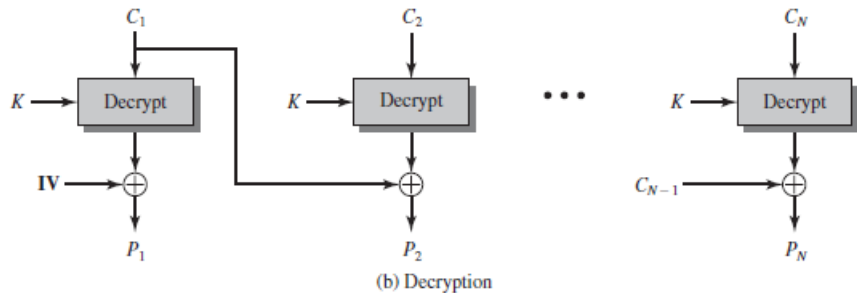
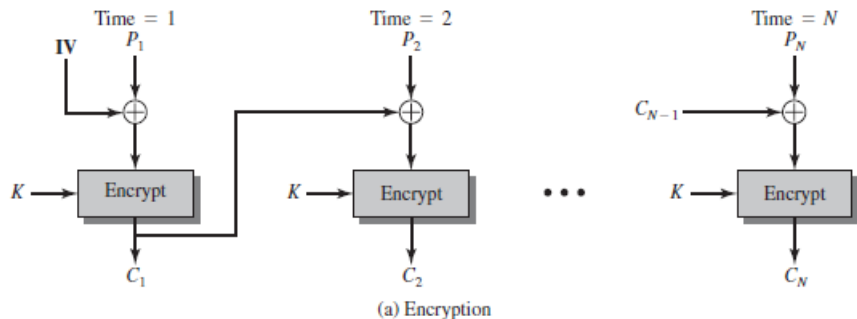
Cipher Block Chaining (CBC) mode decryption

- Ciphertext data is received as blocks $C_1, C_2, \dots, C_n$
- The decryption algorithm now takes three inputs:
 - the key, the ciphertext block and the previous ciphertext block
 - For the first ciphertext block, the same initialisation vector (IV) from the sender side is used
- **Decryption:**
 - $P_1 = D(K, C_1) \oplus IV$
 - $P_i = D(K, C_i) \oplus C_{i-1}$



- Decryption parallelizable?
 - Yes

Cipher Block Chaining(CBC) - Encryption and Decryption



Issues with CBC Mode?

The chaining guards against construction of a code book

- Now if the same plaintext block is encrypted, it may form different ciphertext blocks

What happens if the ciphertext is modified?

- Bitflip error (0 to 1 or vice versa): that block and one bit in the following block will decrypt incorrectly
- If a ciphertext bit (or even a character) is inserted or deleted this will still be detected because of the incorrect ciphertext length
 - But length alone will not be enough to detect if the number of bits inserted/deleted is a multiple of the block size

Cipher Feedback(CFB)

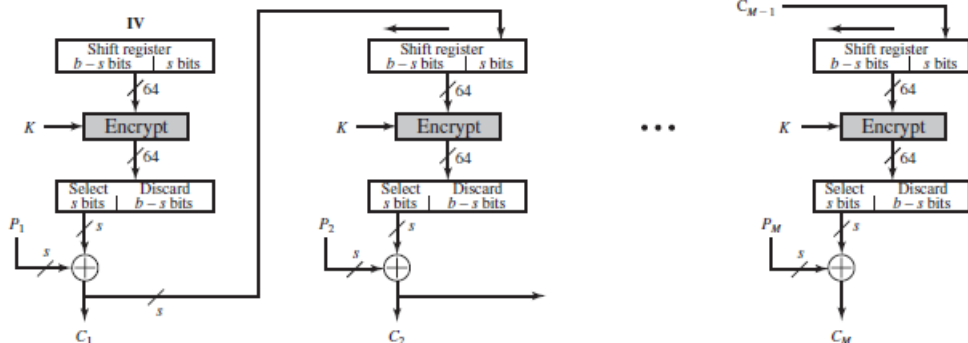
Cipher Feedback (CFB) mode

- Can convert any block cipher into a stream cipher by using the cipher feedback (CFB) mode
- The CFB mode also requires an integer parameter, denoted s , such that $1 \leq s \leq b$
- Assumed that the unit of transmission is s bits; a common value is $s = 8$
- The value of s is sometimes incorporated into the name of the mode, e.g.,
 - the
 - 1-bit CFB mode,
 - the 8-bit CFB mode,
 - the 64-bit CFB mode, or
 - the 128-bit CFB mode
- As with CBC, the units of plaintext are chained together, so that the ciphertext of any plaintext unit is a function of all the preceding plaintext

Cipher Feedback(CFB)

Cipher Feedback (CFB) mode encryption

- The input to the encryption function is a b -bit shift register
 - initially set to some initialization vector (IV)
- The leftmost(most significant) s bits of the output of the encryption function are XORed with the first unit of plaintext P_1 to produce the first unit of ciphertext C_1
- The contents of the shift register are shifted left by s bit and C_1 is placed in the rightmost (least significant) s bits of the shift register

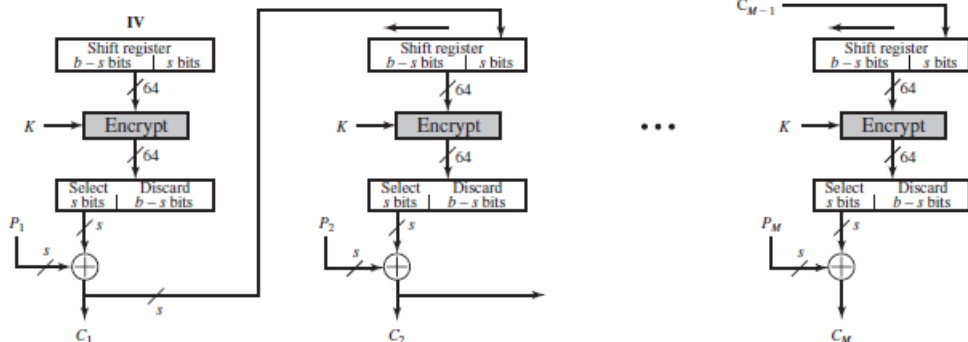


- Encryption parallelizable?

Cipher Feedback(CFB)

Cipher Feedback (CFB) mode encryption

- The input to the encryption function is a b -bit shift register
 - initially set to some initialization vector (IV)
- The leftmost(most significant) s bits of the output of the encryption function are XORed with the first unit of plaintext P_1 to produce the first unit of ciphertext C_1
- The contents of the shift register are shifted left by s bit and C_1 is placed in the rightmost (least significant) s bits of the shift register

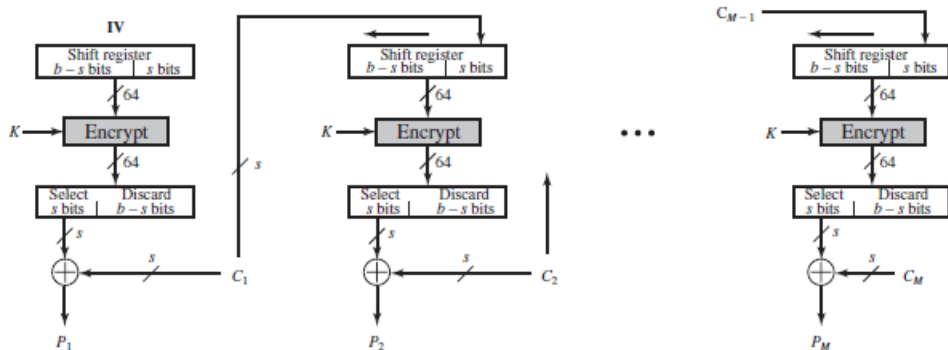


- Encryption parallelizable?
 - No

Cipher Feedback(CFB)

Cipher Feedback (CFB) mode decryption

- The received ciphertext unit is XORed with the output of the encryption function to produce the plaintext unit

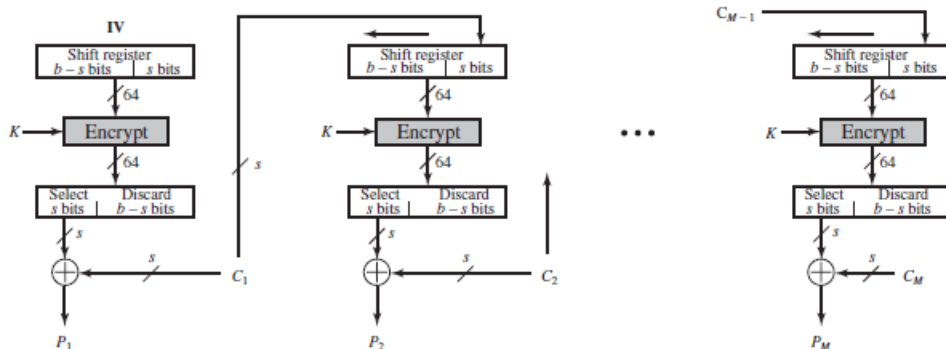


- Decryption parallelizable?

Cipher Feedback(CFB)

Cipher Feedback (CFB) mode decryption

- The received ciphertext unit is XORed with the output of the encryption function to produce the plaintext unit



- Decryption parallelizable?
 - Yes

Issues with CFB Mode?

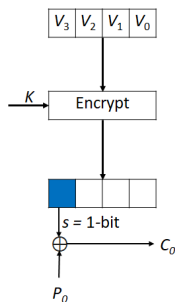
What happens if the ciphertext is modified?

- **Bitflip error (0 to 1 or vice versa):** What happens in the case of stream cipher?
 - What will happen to the case of CFB mode?
- **Bit/Character insertion or deletion:** What happens in the case of stream cipher?
 - What will happen to the case of CFB mode?

CFB Mode - A Hypothetical Example

Consider a hypothetical block cipher with a block size of 4, operating using 1-bit CFB mode, i.e., $s = 1$

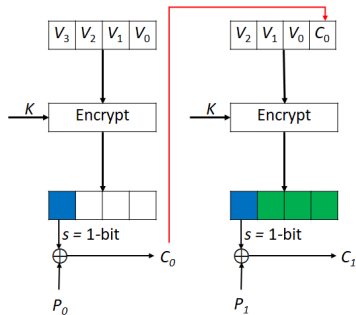
- Consider the encryption of $P = P_0P_1P_2P_3P_4P_5 \dots$
- The initialisation vector is set to $V = V_0V_1V_2V_3$



CFB Mode - A Hypothetical Example

Consider a hypothetical block cipher with a block size of 4, operating using 1-bit CFB mode, i.e., $s = 1$

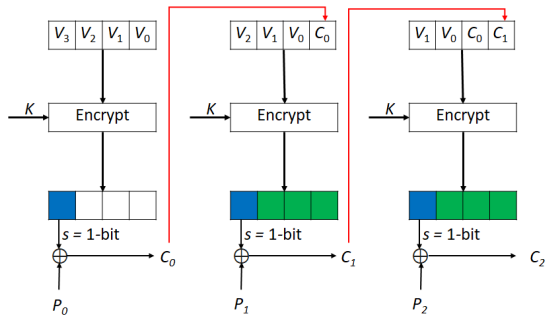
- Consider the encryption of $P = P_0P_1P_2P_3P_4P_5 \dots$
- The initialisation vector is set to $V = V_0V_1V_2V_3$



CFB Mode - A Hypothetical Example

Consider a hypothetical block cipher with a block size of 4, operating using 1-bit CFB mode, i.e., $s = 1$

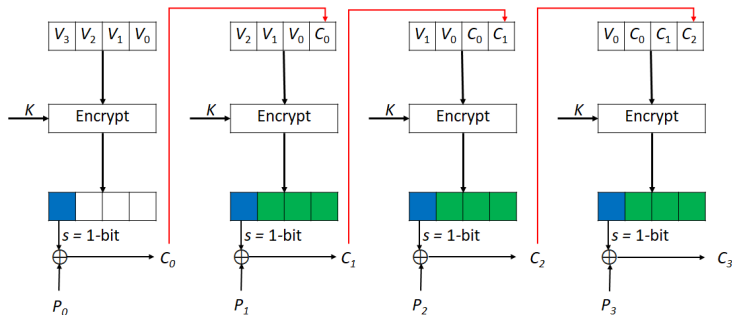
- Consider the encryption of $P = P_0P_1P_2P_3P_4P_5 \dots$
- The initialisation vector is set to $V = V_0V_1V_2V_3$



CFB Mode - A Hypothetical Example

Consider a hypothetical block cipher with a block size of 4, operating using 1-bit CFB mode, i.e., $s = 1$

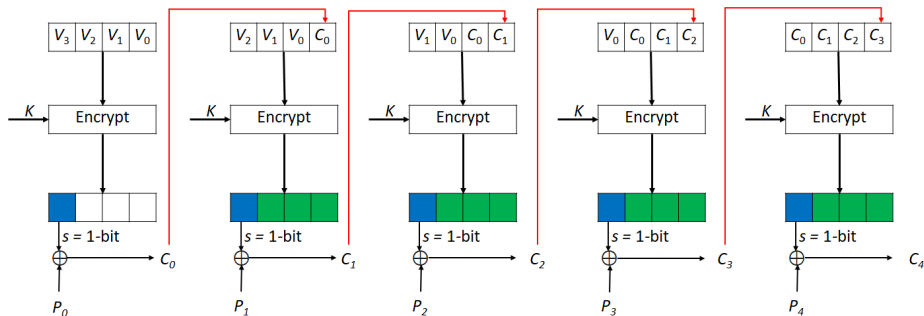
- Consider the encryption of $P = P_0P_1P_2P_3P_4P_5 \dots$
- The initialisation vector is set to $V = V_0V_1V_2V_3$



CFB Mode - A Hypothetical Example

Consider a hypothetical block cipher with a block size of 4, operating using 1-bit CFB mode, i.e., $s = 1$

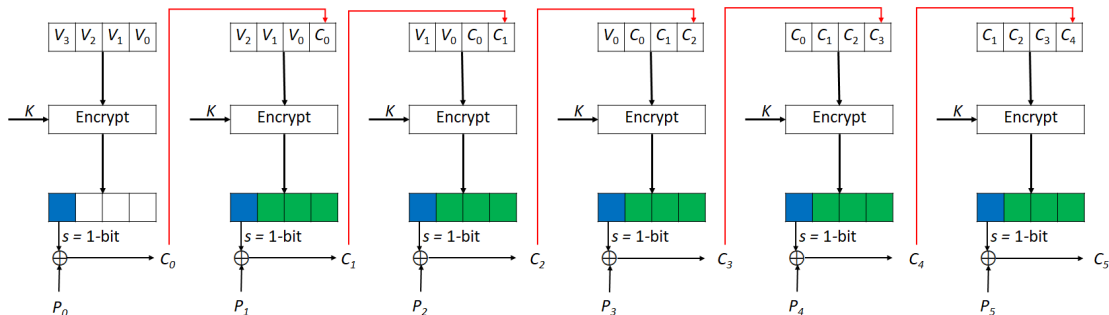
- Consider the encryption of $P = P_0P_1P_2P_3P_4P_5 \dots$
- The initialisation vector is set to $V = V_0V_1V_2V_3$



CFB Mode - A Hypothetical Example

Consider a hypothetical block cipher with a block size of 4, operating using 1-bit CFB mode, i.e., $s = 1$

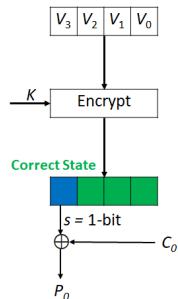
- Consider the encryption of $P = P_0P_1P_2P_3P_4P_5 \dots$
- The initialisation vector is set to $V = V_0V_1V_2V_3$



CFB Mode - A Hypothetical Example

Consider a hypothetical block cipher with a block size of 4, operating using 1-bit CFB mode, i.e., $s = 1$

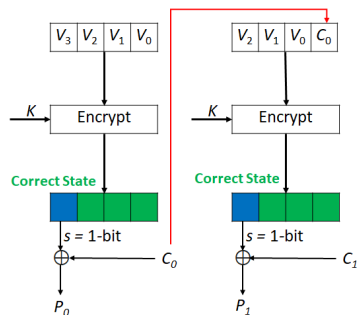
- Decryption of $C = C_0C_1C_2C_3C_4C_5 \dots$
- The initialisation vector is set to $V = V_0V_1V_2V_3$



CFB Mode - A Hypothetical Example

Consider a hypothetical block cipher with a block size of 4, operating using 1-bit CFB mode, i.e., $s = 1$

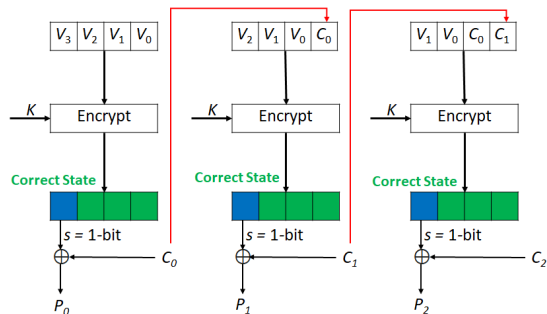
- Decryption of $C = C_0C_1C_2C_3C_4C_5 \dots$
- The initialisation vector is set to $V = V_0V_1V_2V_3$



CFB Mode - A Hypothetical Example

Consider a hypothetical block cipher with a block size of 4, operating using 1-bit CFB mode, i.e., $s = 1$

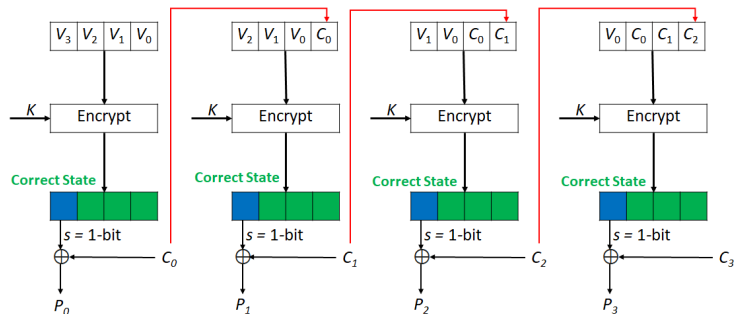
- Decryption of $C = C_0C_1C_2C_3C_4C_5 \dots$
- The initialisation vector is set to $V = V_0V_1V_2V_3$



CFB Mode - A Hypothetical Example

Consider a hypothetical block cipher with a block size of 4, operating using 1-bit CFB mode, i.e., $s = 1$

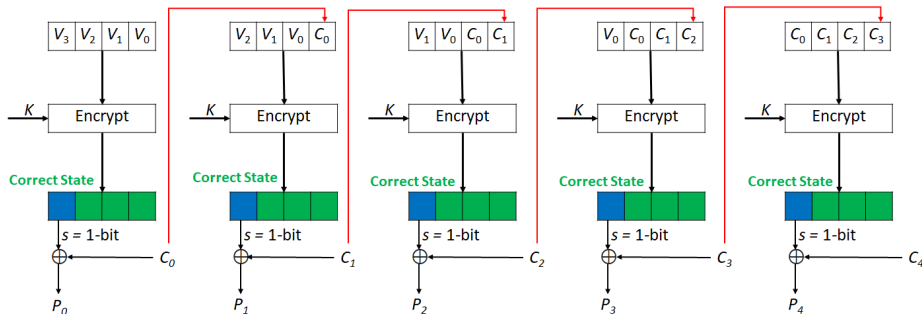
- Decryption of $C = C_0C_1C_2C_3C_4C_5 \dots$
- The initialisation vector is set to $V = V_0V_1V_2V_3$



CFB Mode - A Hypothetical Example

Consider a hypothetical block cipher with a block size of 4, operating using 1-bit CFB mode, i.e., $s = 1$

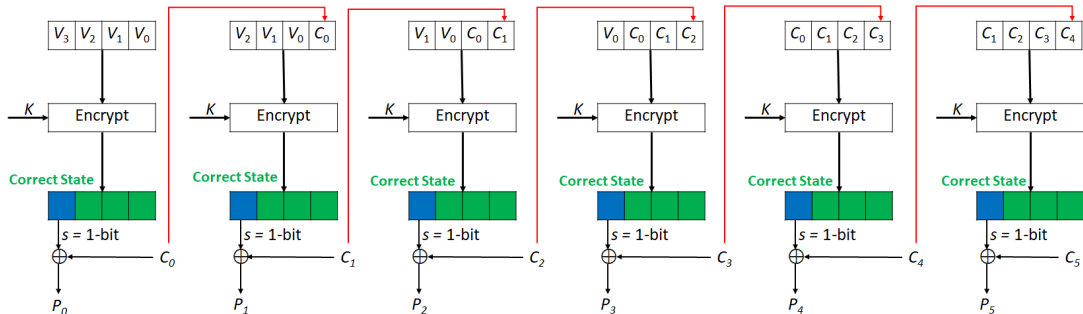
- Decryption of $C = C_0C_1C_2C_3C_4C_5 \dots$
- The initialisation vector is set to $V = V_0V_1V_2V_3$



CFB Mode - A Hypothetical Example

Consider a hypothetical block cipher with a block size of 4, operating using 1-bit CFB mode, i.e., $s = 1$

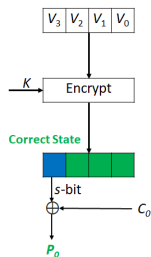
- Decryption of $C = C_0C_1C_2C_3C_4C_5 \dots$
- The initialisation vector is set to $V = V_0V_1V_2V_3$



CFB Mode - Bit Insertion Error

Consider a hypothetical block cipher with a block size of 4, operating using 1-bit CFB mode, i.e., $s = 1$

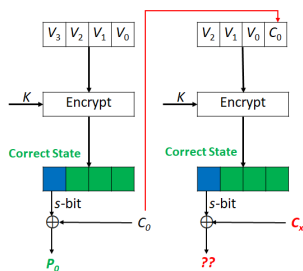
- Decryption of $C' = C_0C_xC_1C_2C_3C_4C_5 \dots$
- The initialisation vector is set to $V = V_0V_1V_2V_3$
- What do you think will happen to the decryption?



CFB Mode - Bit Insertion Error

Consider a hypothetical block cipher with a block size of 4, operating using 1-bit CFB mode, i.e., $s = 1$

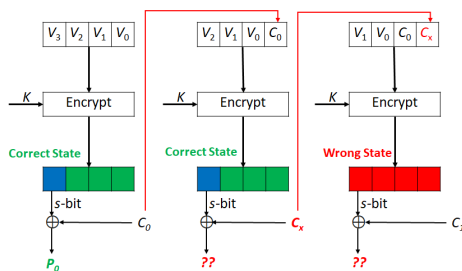
- Decryption of $C' = C_0 C_x C_1 C_2 C_3 C_4 C_5 \dots$
- The initialisation vector is set to $V = V_0 V_1 V_2 V_3$
- What do you think will happen to the decryption?



CFB Mode - Bit Insertion Error

Consider a hypothetical block cipher with a block size of 4, operating using 1-bit CFB mode, i.e., $s = 1$

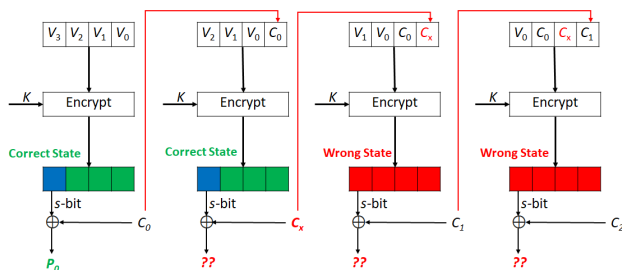
- Decryption of $C' = C_0 \textcolor{red}{C_x} C_1 C_2 C_3 C_4 C_5 \dots$
- The initialisation vector is set to $V = V_0 V_1 V_2 V_3$
- **What do you think will happen to the decryption?**



CFB Mode - Bit Insertion Error

Consider a hypothetical block cipher with a block size of 4, operating using 1-bit CFB mode, i.e., $s = 1$

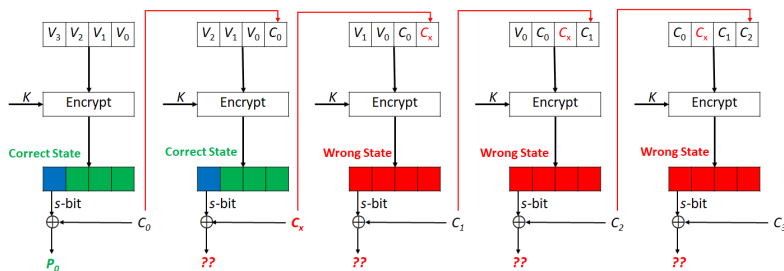
- Decryption of $C' = C_0 \textcolor{red}{C_x} C_1 C_2 C_3 C_4 C_5 \dots$
- The initialisation vector is set to $V = V_0 V_1 V_2 V_3$
- **What do you think will happen to the decryption?**



CFB Mode - Bit Insertion Error

Consider a hypothetical block cipher with a block size of 4, operating using 1-bit CFB mode, i.e., $s = 1$

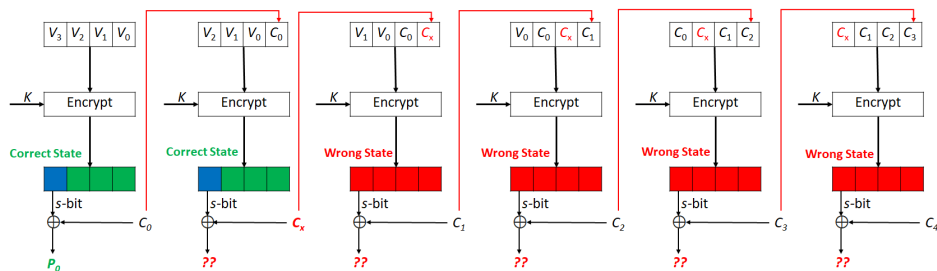
- Decryption of $C' = C_0 \textcolor{red}{C_x} C_1 C_2 C_3 C_4 C_5 \dots$
- The initialisation vector is set to $V = V_0 V_1 V_2 V_3$
- **What do you think will happen to the decryption?**



CFB Mode - Bit Insertion Error

Consider a hypothetical block cipher with a block size of 4, operating using 1-bit CFB mode, i.e., $s = 1$

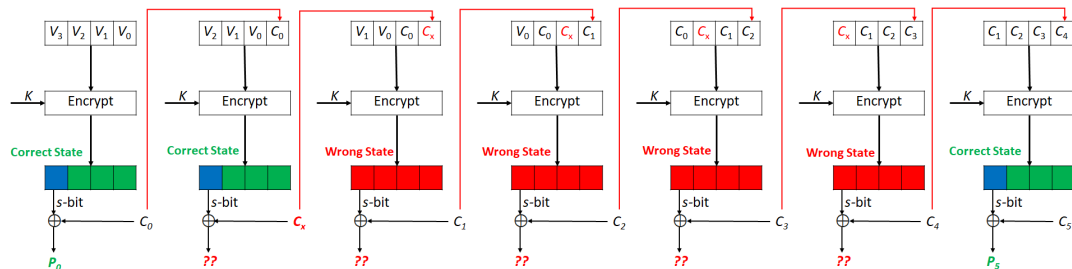
- Decryption of $C' = C_0 \textcolor{red}{C_x} C_1 C_2 C_3 C_4 C_5 \dots$
- The initialisation vector is set to $V = V_0 V_1 V_2 V_3$
- **What do you think will happen to the decryption?**



CFB Mode - Bit Insertion Error

Consider a hypothetical block cipher with a block size of 4, operating using 1-bit CFB mode, i.e., $s = 1$

- Decryption of $C' = C_0 \textcolor{red}{C_x} C_1 C_2 C_3 C_4 C_5 \dots$
- The initialisation vector is set to $V = V_0 V_1 V_2 V_3$
- **What do you think will happen to the decryption?**



CFB Mode - Bit Deletion Error

What about bit deletion?

Consider a hypothetical block cipher with a block size of 4, operating using 1-bit CFB mode, i.e., $s = 1$

- Original ciphertext $C = C_0C_1C_2C_3C_4C_5 \dots$
- The ciphertext has been modified by deleting C_0
- Decryption of $C' = C_1C_2C_3C_4C_5 \dots$
- The initialisation vector is set to $V = V_0V_1V_2V_3$

What do you think will happen to the decryption?

CFB Mode - Bit Insertion/Deletion Error

Will the synchronization always be restored in the case of bit insertion/deletion error in CFB mode?

- What happens if the bit insertion/deletion is a multiple of the transmission unit s ?
- What about if the bit insertion/deletion is not a multiple of transmission unit s ?

Output Feedback(OFB)

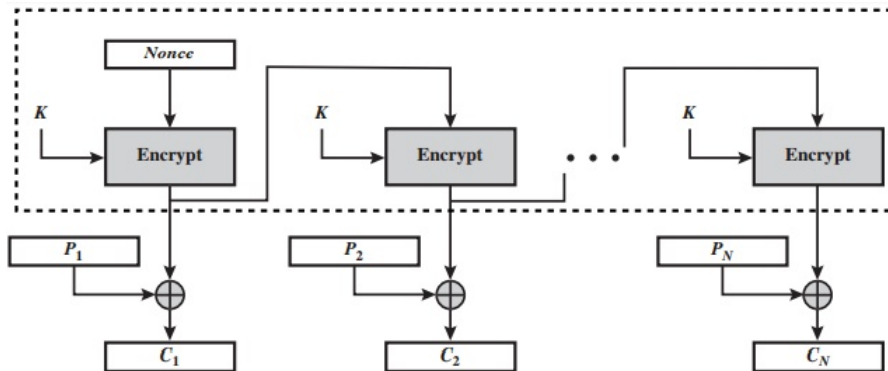
Output Feedback (OFB) mode

- Similar in structure to that of CFB
- The output of the encryption function that is fed back to the shift register in OFB, whereas in CFB, the ciphertext unit is fed back to the shift register
- The other difference is that the OFB mode operates on full blocks of plaintext and ciphertext, not on an s -bit subset

Output Feedback(OFB)

Output Feedback (OFB) mode

- The output of the encryption function that is fed back to the shift register in OFB, whereas in CFB, the ciphertext unit is fed back to the shift register
- The OFB mode requires that the IV is a nonce
- Compute the output of the encryption function and XOR it with the plaintext to encrypt it

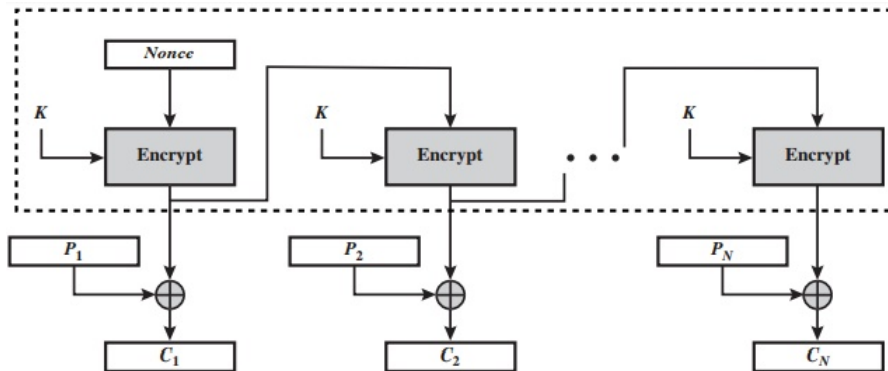


- Encryption parallelizable?

Output Feedback(OFB)

Output Feedback (OFB) mode

- The output of the encryption function that is fed back to the shift register in OFB, whereas in CFB, the ciphertext unit is fed back to the shift register
- The OFB mode requires that the IV is a nonce
- Compute the output of the encryption function and XOR it with the plaintext to encrypt it

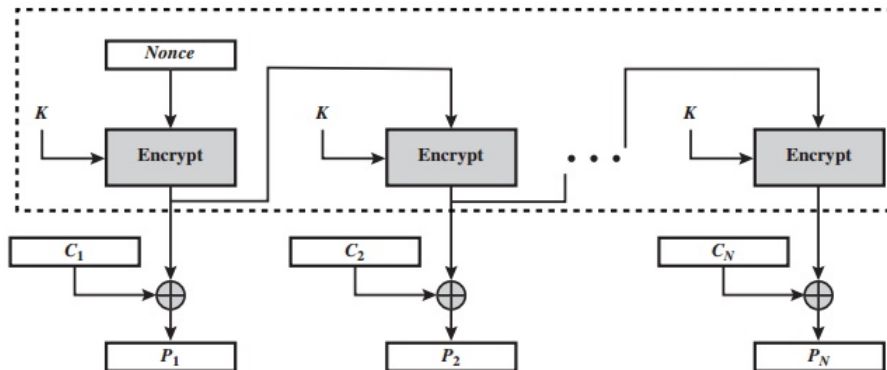


- Encryption parallelizable?
 - No

Output Feedback(OFB)

Output Feedback (OFB) mode

- Because of the symmetry of the XOR operation, encryption and decryption are exactly the same
- Start with the same nonce and key at the receiver
- Compute the output of the encryption function and XOR it with the ciphertext to decrypt it

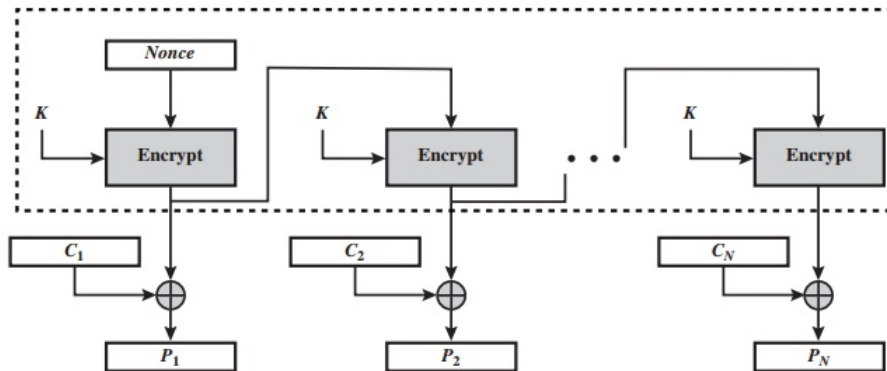


- Decryption parallelizable?

Output Feedback(OFB)

Output Feedback (OFB) mode

- Because of the symmetry of the XOR operation, encryption and decryption are exactly the same
- Start with the same nonce and key at the receiver
- Compute the output of the encryption function and XOR it with the ciphertext to decrypt it



- Decryption parallelizable?
 - No

Output Feedback(OFB)

What happens if there is an error in the transmission?

- Bit flip error?

Output Feedback(OFB)

What happens if there is an error in the transmission?

- Bit flip error?
 - Bit flip in the ciphertext does not propagate
- Bit insertion/deletion error?

Output Feedback(OFB)

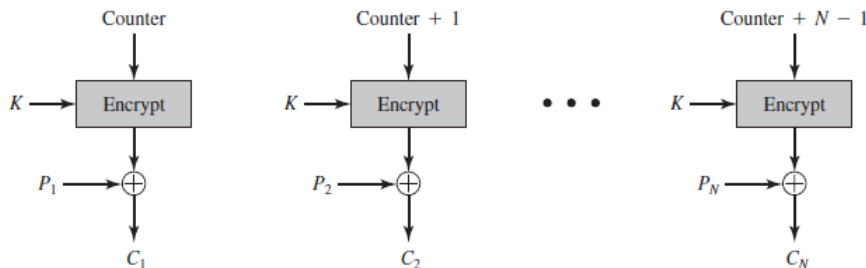
What happens if there is an error in the transmission?

- Bit flip error?
 - Bit flip in the ciphertext does not propagate
- Bit insertion/deletion error?
 - Should be able to detect due to the incorrect length?
 - How about if inserted/deleted in multiples of the block size?
 - Synchronization lost?
 - Recover synchronization like CFB mode?

Counter Mode

Counter Mode Encryption

- A counter is equal to the plaintext block size
- Typically, the counter is initialized to some value and then incremented by 1 for each subsequent block
- The counter is encrypted and then XORed with the plaintext block to produce the ciphertext block

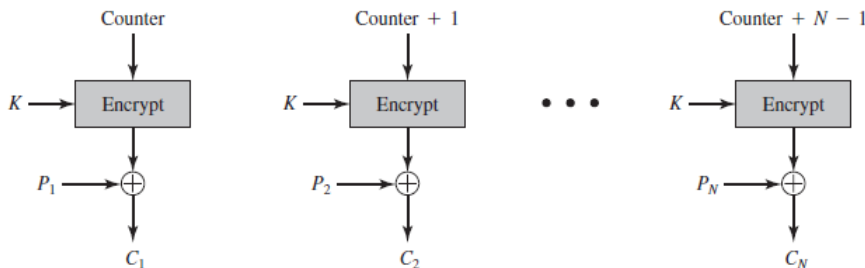


- Encryption parallelizable?

Counter Mode

Counter Mode Encryption

- A counter is equal to the plaintext block size
- Typically, the counter is initialized to some value and then incremented by 1 for each subsequent block
- The counter is encrypted and then XORed with the plaintext block to produce the ciphertext block

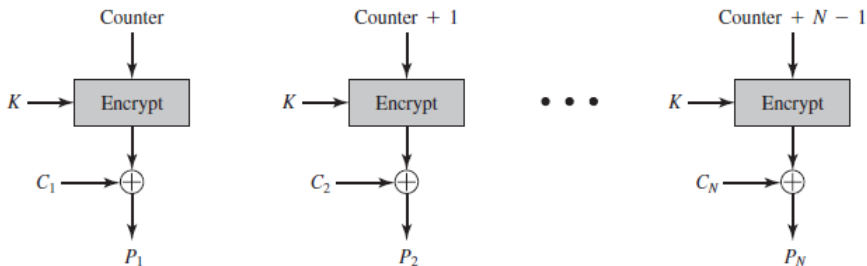


- Encryption parallelizable?
 - Yes

Counter Mode

Counter Mode Decryption

- The same sequence of counter values is used
- The counter is encrypted and then XORed with the ciphertext block to produce the plaintext block

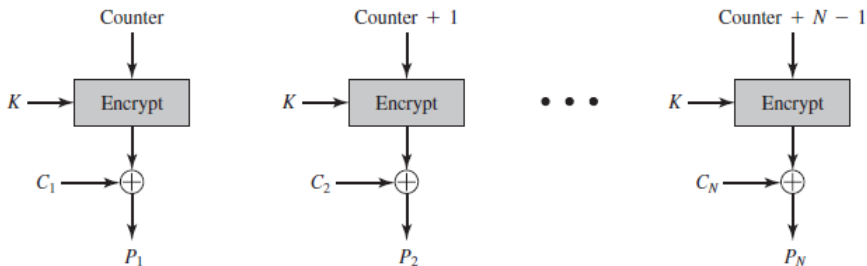


- Decryption parallelizable?

Counter Mode

Counter Mode Decryption

- The same sequence of counter values is used
- The counter is encrypted and then XORed with the ciphertext block to produce the plaintext block



- Decryption parallelizable?
 - Yes

Counter mode

What happens if there is an error in the transmission?

- Bit flip error in the ciphertext?
- Bit insertion/deletion error in the ciphertext?

CTR mode has similar characteristics to OFB, but also allows a random access property during decryption

- well suited to operate on a multi-processor machine where blocks can be encrypted in parallel

Applications of Block Ciphers

Block ciphers often used for providing confidentiality services for applications involving processing large volumes of data where time delays are not critical.

- Computer files
 - Almost all modern file and disk encryption systems use AES, including Windows' Bitlocker, Mac OS' FileVault, Linux' dm-crypt
- Databases
 - Adobe AIR database encryption uses AES in Counter with CBC-MAC (CCM) mode
- Email messages
 - Lots of programs use AES

End of lecture 7! 😊

Summary

- Block cipher is one type of symmetric cryptographic primitive
- Can be used in different modes of operations
- Suitable for applications where time delays are not critical

Next lecture

- Symmetric Cryptography - Integrity assurance and message authentication codes

Additional materials: NIST standard on block cipher mode of operations

- <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38a.pdf>

Acknowledgement

- Some of the contents on AES are adopted from the below slide:
http://www.cse.iitm.ac.in/~chester/courses/16e_cns/slides/04_BlockCiphers.pdf

Announcements

- Individual assignment for SWE308 has been uploaded in Moodle
- Mid-semester exam – see Moodle for details
-

QUESTIONS?

Appendix: University's vision and mission

- ① Vision: Xiamen University Malaysia aspires to become a university with a distinct global outlook, featuring first-class teaching and research, and embracing cultural diversity.
- ② Mission: To nurture young talents with dignity and wisdom, turning them into fine citizens of the region who will contribute to the prosperity of the people and social progress of Malaysia, China and Southeast Asia.